



Machine Learning

Lab - 1

Dhruv Kakkad

Eno:24031701022

Topic: Basic Array Manipulation

1. Import Libraries: Import the numpy library.

```
In [2]: import numpy as np
```

2. Create Arrays: Create an array of 10 zeros.

```
In [2]: np.zeros(10)
```

```
Out[2]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

3. Create Arrays: Create an array of 10 ones.

```
In [6]: np.ones(10)
```

```
Out[6]: array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

4. Create Arrays: Create an array of 10 fives.

```
In [24]: np.full(10,5)
```

```
Out[24]: array([5, 5, 5, 5, 5, 5, 5, 5, 5, 5])
```

5. Range:

Create an array of the integers from 10 to 50.

```
In [9]: np.arange(10,51)
```

```
Out[9]: array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26,
              27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
              44, 45, 46, 47, 48, 49, 50])
```

6. Range (Step):

Create an array of all the even integers from 10 to 50.

```
In [10]: np.arange(10,51,2)
```

```
Out[10]: array([10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42,
               44, 46, 48, 50])
```

7. Matrix Creation:

Create a 3x3 matrix with values ranging from 0 to 8.

```
In [12]: np.arange(0,9).reshape(3,3)
```

```
Out[12]: array([[0, 1, 2],
               [3, 4, 5],
               [6, 7, 8]])
```

8. Identity Matrix:

Create a 3x3 identity matrix.

```
In [13]: np.eye(3,3)
```

```
Out[13]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

9. Random Numbers:

Use NumPy to generate a random number between 0 and 1.

```
In [20]: np.random.random()
```

```
Out[20]: 0.5195141508670106
```

10. Random Distribution:

Use NumPy to generate an array of 25 random numbers sampled from a standard normal distribution.

```
In [3]: np.random.randn(25)
```

```
Out[3]: array([-2.24206674, -0.2201649 , -0.90252365, -1.58955782, -0.65907496,  
              -1.49844026,  0.03389392, -0.43899448,  0.51598496,  1.01041189,  
              -0.81731041, -0.90824947, -0.75955582,  0.36151996, -0.74010951,  
               0.72958255, -1.75961435, -0.60066709, -0.12794238, -1.29270334,  
              -0.24762583, -0.03112366,  1.86131862, -0.16181443, -0.18300703])
```

11. Matrix Steps:

Create the specific matrix ranging from 0.01 to 1.0 with a step of 0.01.

```
In [6]: np.arange(0.01,1.0,0.01)
```

```
Out[6]: array([0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1 , 0.11,  
              0.12, 0.13, 0.14, 0.15, 0.16, 0.17, 0.18, 0.19, 0.2 , 0.21, 0.22,  
              0.23, 0.24, 0.25, 0.26, 0.27, 0.28, 0.29, 0.3 , 0.31, 0.32, 0.33,  
              0.34, 0.35, 0.36, 0.37, 0.38, 0.39, 0.4 , 0.41, 0.42, 0.43, 0.44,  
              0.45, 0.46, 0.47, 0.48, 0.49, 0.5 , 0.51, 0.52, 0.53, 0.54, 0.55,  
              0.56, 0.57, 0.58, 0.59, 0.6 , 0.61, 0.62, 0.63, 0.64, 0.65, 0.66,  
              0.67, 0.68, 0.69, 0.7 , 0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77,  
              0.78, 0.79, 0.8 , 0.81, 0.82, 0.83, 0.84, 0.85, 0.86, 0.87, 0.88,  
              0.89, 0.9 , 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98, 0.99])
```

12. Linspace:

Create an array of 20 linearly spaced points between 0 and 1.

```
In [8]: np.linspace(0,1,20)
```

```
Out[8]: array([0.          , 0.05263158, 0.10526316, 0.15789474, 0.21052632,  
              0.26315789, 0.31578947, 0.36842105, 0.42105263, 0.47368421,  
              0.52631579, 0.57894737, 0.63157895, 0.68421053, 0.73684211,  
              0.78947368, 0.84210526, 0.89473684, 0.94736842, 1.          ])
```

13. Indexing:

Use numpy indexing to extract specific chunks from a matrix (e.g., rows 2-4, columns 1-3).

```
In [22]: arr = np.arange(1,26).reshape(5,5)  
arr
```

```
Out[22]: array([[ 1,  2,  3,  4,  5],  
               [ 6,  7,  8,  9, 10],  
               [11, 12, 13, 14, 15],  
               [16, 17, 18, 19, 20],  
               [21, 22, 23, 24, 25]])
```

```
In [18]: arr[2:,1:4]
```

```
Out[18]: array([[12, 13, 14],  
               [17, 18, 19],  
               [22, 23, 24]])
```

14. Summation: Get the sum of all the values in a matrix.

```
In [19]: arr.sum()
```

```
Out[19]: np.int64(325)
```

15. Standard Deviation:

Get the standard deviation of the values in a matrix.

```
In [20]: arr.std()
```

```
Out[20]: np.float64(7.211102550927978)
```

16. Column Sums: Get the sum of all the columns in a matrix.

```
In [23]: np.sum(arr,axis=0)
```

```
Out[23]: array([55, 60, 65, 70, 75])
```